

(1)对问题理解后，写明解决思路，并使用穷举法解决问题：先使用非逻辑编程的方式穷举，再对结果使用逻辑编程的方式筛选。

1. 问题分析与解决思路

八皇后问题要求在 8×8 的国际象棋棋盘上放置 8 个皇后，使得任意两个皇后都不能处于同一行、同一列或同一条对角线上。本题要求先使用非逻辑编程的方式穷举，再对结果使用逻辑编程的方式进行筛选。

直觉上，可以使用二维数组 `board[8][8]` 来表示棋盘，其中某个位置为 1 表示该位置放置皇后，为 0 表示该位置为空。但是经过分析可以发现，八皇后问题的核心约束并不依赖二维数组本身，而主要依赖皇后的行号和列号之间的关系。因此，为了简化问题表示，本题不直接使用二维数组 `board` 进行求解，而是使用一维数组 `placement` 表示一个棋盘布局。

设：

`placement[row] = col`

表示第 `row` 行的皇后放在第 `col` 列。这样，“每一行放置一个皇后”由约束便能天然保证了。

本小题要求先使用非逻辑编程方式穷举，再使用逻辑编程方式筛选结果。因此，第一阶段先用普通的回溯算法生成 $0 \sim N-1$ 的所有全排列，每个排列作为一个候选布局 `placement`。

由于全排列中每一项（每个皇后的列坐标）不重复，所以这些候选布局也已经满足“每列一个皇后”的基本约束。

第二阶段再对第一阶段生成的所有全排列中的每个候选布局进行逻辑编程筛选，判断任意两个皇后是否满足“不在同一条对角线”这一约束。筛选结果即为八皇后问题的合法解。

2. 非逻辑式编程：穷举所有解

由于八皇后问题要求 8 个皇后不能处于同一列，因此本阶段生成 $1 \sim 8$ 的所有全排列。每一个排列都表示一种满足“每行一个皇后、每列一个皇后”的候选布局。所有候选布局组成集合 `candidates`。

本阶段只负责生成候选解空间，不判断对角线冲突。对角线约束将在下一阶段使用逻辑编程方式进行筛选。

```
In [2]: N = 8      #棋盘大小
nums = list(range(1, N + 1))    #[1, 2, 3, 4, 5, 6, 7, 8]
candidates = []

def permutations(nums, k):
    if k == N:
        # 当 k == N 时，说明已经得到一个完整排列。
        # 将其复制添加到 candidates 中
        candidates.append(nums[:])
```

```

        return

    for i in range(k, N):
        # 将 nums[i] 放到当前位置 k
        nums[k], nums[i] = nums[i], nums[k]

        # 递归生成 k + 1 之后位置的全排列
        permutations(nums, k + 1)

        # 撤销选择, 回到交换前的状态
        nums[k], nums[i] = nums[i], nums[k]

    permutations(nums, 0)

    len(candidates)

```

Out[2]: 40320

结果为 $8! = 40320$, 说明正确

3. 逻辑编程：筛选

上一阶段已生成所有可能的皇后布局，本阶段使用 Kanren 的逻辑编程思想对候选它们进行筛选。筛选规则是：任意两个皇后不能处于同一条对角线上。

对于第 `row1` 行和第 `row2` 行的两个皇后，若满足：

```
abs(row1 - row2) == abs(col1 - col2)
```

则它们位于同一条对角线，需要排除该布局。

In [3]: `%pip install kanren`

```
Collecting kanren
  Downloading kanren-0.3.0-py3-none-any.whl.metadata (6.2 kB)
Collecting toolz (from kanren)
  Downloading toolz-1.1.0-py3-none-any.whl.metadata (5.1 kB)
Collecting multipledispatch (from kanren)
  Downloading multipledispatch-1.0.0-py3-none-any.whl.metadata (3.8 kB)
Collecting unification>=0.3.1 (from kanren)
  Downloading unification-0.3.1-py3-none-any.whl.metadata (2.2 kB)
Downloading kanren-0.3.0-py3-none-any.whl (17 kB)
Downloading unification-0.3.1-py3-none-any.whl (7.0 kB)
Downloading multipledispatch-1.0.0-py3-none-any.whl (12 kB)
Downloading toolz-1.1.0-py3-none-any.whl (58 kB)
Installing collected packages: unification, multipledispatch, toolz, kanren
```

```
----- 1/4 [multipledispatch]
----- 2/4 [toolz]
----- 2/4 [toolz]
----- 2/4 [toolz]
----- 2/4 [toolz]
----- 2/4 [toolz]
----- 2/4 [toolz]
----- 2/4 [toolz]
----- 3/4 [kanren]
----- 3/4 [kanren]
----- 4/4 [kanren]
```

Successfully installed kanren-0.3.0 multipledispatch-1.0.0 toolz-1.1.0 unification-0.3.1

Note: you may need to restart the kernel to use updated packages.

```
In [7]: from kanren import run, var, eq
        #run 用于发起逻辑查询, var() 用于声明逻辑变量, eq() 用于声明相等关系。
```

这里为了对已经生成的候选布局进行筛选，将“是否满足对角线约束”封装为一个逻辑查询：若布局合法，则查询成功；否则查询失败。

定义筛选函数：

```
In [8]: def diagonal_safe(placement):
        """
        判断一个 placement 是否满足对角线不冲突约束。

        placement[row] = col
        其中 row 从 0 开始, col 使用 1 ~ 8。
        """
        for row1 in range(N):
            for row2 in range(row1 + 1, N):
                col1 = placement[row1]
                col2 = placement[row2]

                if abs(row1 - row2) == abs(col1 - col2):
                    return False

        return True

    def logic_filter(placement):
        """
        使用 Kanren 的逻辑查询形式筛选候选布局。

        若 placement 满足对角线约束, 则令 result == True;
```

```
    否则令 result == False。  
    最后查询 result 是否可以等于 True。  
    """"  
    result = var()  
  
    return len(run(1, result, eq(result, diagonal_safe(placement)), eq(result, T
```

筛选:

```
In [11]: solutions = []  
  
for placement in candidates:  
    if logic_filter(placement):  
        solutions.append(placement)  
  
solutions
```

```
Out[11]: [[1, 5, 8, 6, 3, 7, 2, 4],
[1, 6, 8, 3, 7, 4, 2, 5],
[1, 7, 4, 6, 8, 2, 5, 3],
[1, 7, 5, 8, 2, 4, 6, 3],
[2, 4, 6, 8, 3, 1, 7, 5],
[2, 5, 7, 4, 1, 8, 6, 3],
[2, 5, 7, 1, 3, 8, 6, 4],
[2, 6, 1, 7, 4, 8, 3, 5],
[2, 6, 8, 3, 1, 4, 7, 5],
[2, 7, 3, 6, 8, 5, 1, 4],
[2, 7, 5, 8, 1, 4, 6, 3],
[2, 8, 6, 1, 3, 5, 7, 4],
[3, 1, 7, 5, 8, 2, 4, 6],
[3, 5, 2, 8, 1, 7, 4, 6],
[3, 5, 2, 8, 6, 4, 7, 1],
[3, 5, 7, 1, 4, 2, 8, 6],
[3, 5, 8, 4, 1, 7, 2, 6],
[3, 6, 4, 1, 8, 5, 7, 2],
[3, 6, 4, 2, 8, 5, 7, 1],
[3, 6, 2, 5, 8, 1, 7, 4],
[3, 6, 2, 7, 5, 1, 8, 4],
[3, 6, 2, 7, 1, 4, 8, 5],
[3, 6, 8, 2, 4, 1, 7, 5],
[3, 6, 8, 1, 5, 7, 2, 4],
[3, 6, 8, 1, 4, 7, 5, 2],
[3, 7, 2, 8, 5, 1, 4, 6],
[3, 7, 2, 8, 6, 4, 1, 5],
[3, 8, 4, 7, 1, 6, 2, 5],
[4, 2, 5, 8, 6, 1, 3, 7],
[4, 2, 7, 5, 1, 8, 6, 3],
[4, 2, 7, 3, 6, 8, 1, 5],
[4, 2, 7, 3, 6, 8, 5, 1],
[4, 2, 8, 5, 7, 1, 3, 6],
[4, 2, 8, 6, 1, 3, 5, 7],
[4, 1, 5, 8, 6, 3, 7, 2],
[4, 1, 5, 8, 2, 7, 3, 6],
[4, 6, 1, 5, 2, 8, 3, 7],
[4, 6, 8, 2, 7, 1, 3, 5],
[4, 6, 8, 3, 1, 7, 5, 2],
[4, 7, 3, 8, 2, 5, 1, 6],
[4, 7, 1, 8, 5, 2, 6, 3],
[4, 7, 5, 3, 1, 6, 8, 2],
[4, 7, 5, 2, 6, 1, 3, 8],
[4, 8, 1, 3, 6, 2, 7, 5],
[4, 8, 1, 5, 7, 2, 6, 3],
[4, 8, 5, 3, 1, 7, 2, 6],
[5, 2, 4, 6, 8, 3, 1, 7],
[5, 2, 4, 7, 3, 8, 6, 1],
[5, 2, 6, 1, 7, 4, 8, 3],
[5, 2, 8, 1, 4, 7, 3, 6],
[5, 3, 1, 6, 8, 2, 4, 7],
[5, 3, 1, 7, 2, 8, 6, 4],
[5, 3, 8, 4, 7, 1, 6, 2],
[5, 1, 4, 6, 8, 2, 7, 3],
[5, 1, 8, 4, 2, 7, 3, 6],
[5, 1, 8, 6, 3, 7, 2, 4],
[5, 7, 4, 1, 3, 8, 6, 2],
[5, 7, 1, 4, 2, 8, 6, 3],
[5, 7, 1, 3, 8, 6, 4, 2],
[5, 7, 2, 4, 8, 1, 3, 6],
```

```
[5, 7, 2, 6, 3, 1, 4, 8],
[5, 7, 2, 6, 3, 1, 8, 4],
[5, 8, 4, 1, 3, 6, 2, 7],
[5, 8, 4, 1, 7, 2, 6, 3],
[6, 2, 7, 1, 4, 8, 5, 3],
[6, 2, 7, 1, 3, 5, 8, 4],
[6, 3, 5, 7, 1, 4, 2, 8],
[6, 3, 5, 8, 1, 4, 2, 7],
[6, 3, 1, 7, 5, 8, 2, 4],
[6, 3, 1, 8, 5, 2, 4, 7],
[6, 3, 1, 8, 4, 2, 7, 5],
[6, 3, 7, 4, 1, 8, 2, 5],
[6, 3, 7, 2, 4, 8, 1, 5],
[6, 3, 7, 2, 8, 5, 1, 4],
[6, 4, 2, 8, 5, 7, 1, 3],
[6, 4, 1, 5, 8, 2, 7, 3],
[6, 4, 7, 1, 3, 5, 2, 8],
[6, 4, 7, 1, 8, 2, 5, 3],
[6, 1, 5, 2, 8, 3, 7, 4],
[6, 8, 2, 4, 1, 7, 5, 3],
[7, 2, 4, 1, 8, 5, 3, 6],
[7, 2, 6, 3, 1, 4, 8, 5],
[7, 3, 1, 6, 8, 5, 2, 4],
[7, 3, 8, 2, 5, 1, 6, 4],
[7, 4, 2, 5, 8, 1, 3, 6],
[7, 4, 2, 8, 6, 1, 3, 5],
[7, 5, 3, 1, 6, 8, 2, 4],
[7, 1, 3, 8, 6, 4, 2, 5],
[8, 2, 4, 1, 7, 5, 3, 6],
[8, 2, 5, 3, 1, 7, 4, 6],
[8, 3, 1, 6, 2, 5, 7, 4],
[8, 4, 1, 3, 6, 2, 7, 5]]
```

用一个函数来打印布局：验证是否正确：

```
In [13]: def print_placement(placement):
        """
        以文本形式打印一个八皇后布局。
        Q 表示皇后，. 表示空格。
        """
        for col in placement:
            row_text = [". " for _ in range(N)]
            row_text[col - 1] = "Q"
            print(" ".join(row_text))

print_placement(solutions[0])
```

```
Q . . . . .
. . . . Q . .
. . . . . . Q
. . . . . Q .
. . Q . . . .
. . . . . Q .
. Q . . . . .
. . . Q . . .
```

(2) 在(1)的基础上，不使用穷举法解决问题，而只使用逻辑编程。

在上一小题中，我先用非逻辑编程生成候选布局 `candidates`，再进行筛选。

本小题不再预先生成候选解，而是直接使用 Kanren 声明逻辑变量和约束，让逻辑查询过程自动搜索满足条件的布局。

仍然使用一维数组 `placement` 表示棋盘状态：

```
placement[row] = col
```

其中 `row` 表示皇后所在行，`col` 表示皇后所在列。

1. 声明逻辑变量

`placement` 是一个由 8 个逻辑变量组成的元组。每个逻辑变量表示对应行皇后所在的列号。

```
In [20]: cols = tuple(nums)

placement = tuple(var() for _ in range(N))

placement
```

```
Out[20]: (~_120969,
~_120970,
~_120971,
~_120972,
~_120973,
~_120974,
~_120975,
~_120976)
```

2. 定义安全约束

对于当前行 `row` 的皇后，只需要检查它是否与之前各行的皇后冲突。

冲突包括两种情况：

1. 位于同一列；
2. 位于同一条对角线。

```
In [21]: from kanren import membero, reify
```

```
In [22]: def safe_with_previouslyo(placement, row):
        """
        Kanren 目标函数：
        判断第 row 行皇后是否与之前各行皇后冲突。
        """
        def goal(state):
            values = reify(placement[:row + 1], state)
            current_col = values[row]

            for prev_row in range(row):
                prev_col = values[prev_row]

                # 同列冲突
                if current_col == prev_col:
                    return iter(())
```

```

        # 对角线冲突
        if abs(row - prev_row) == abs(current_col - prev_col):
            return iter(())

    return iter((state,))

return goal

```

3. 构造逻辑约束

对每一行皇后添加两个约束：

1. 该行皇后的列号必须属于 1 ~ 8 ；
2. 该行皇后不能与之前各行皇后冲突。

```

In [23]: goals = []

for row in range(N):
    goals.append(membero(placement[row], cols))
    goals.append(safe_with_previouso(placement, row))

```

4. 执行逻辑查询

这里不再使用上一小题中的 `candidates` ，而是直接由 Kanren 搜索满足所有约束的 `placement` 。

```

In [24]: logic_solutions = run(0, placement, *goals)
len(logic_solutions)

```

Out[24]: 92

```

In [25]: print_placement(logic_solutions[0])

```

```

Q . . . . .
. . . . Q . .
. . . . . . Q
. . . . . Q .
. . Q . . . .
. . . . . Q .
. Q . . . . .
. . . Q . . .

```

(3) 对Kanren 库未实现的函数，如`neq`函数，进行实现，并用于解决八皇后问题。并比较不同方法的运算时间、算法复杂度。

1. 实现 `neq` 函数

```

In [26]: def neq(x, y):
        """
        Kanren 逻辑目标函数：
        约束 x 和 y 不相等。

        若 x != y, 则目标成功；
        若 x == y, 则目标失败。
        """

```

```

"""
def goal(state):
    x_value = reify(x, state)
    y_value = reify(y, state)

    if x_value == y_value:
        return iter(())

    return iter((state,))

return goal

```

2. 使用 `neq` 解决八皇后问题

```

In [27]: def diagonal_neq(row1, col1, row2, col2):
        """
        Kanren 目标函数：
        约束第 row1 行和第 row2 行的两个皇后不在同一条对角线上。
        """
        def goal(state):
            c1 = reify(col1, state)
            c2 = reify(col2, state)

            row_distance = abs(row1 - row2)
            col_distance = abs(c1 - c2)

            # 不在同一对角线，即行差 != 列差
            return neq(row_distance, col_distance)(state)

        return goal

```

```

In [30]: placement_neq = tuple(var() for _ in range(N))

goals_neq = []

for row in range(N):
    # 当前行皇后的列号必须在 1 ~ 8 中
    goals_neq.append(membero(placement_neq[row], nums))

    # 当前行只需要和之前各行比较
    for prev_row in range(row):
        # 不同列
        goals_neq.append(neq(placement_neq[row], placement_neq[prev_row]))

        # 不同对角线
        goals_neq.append(
            diagonal_neq(
                row,
                placement_neq[row],
                prev_row,
                placement_neq[prev_row]
            )
        )

```

```

In [31]: solutions_neq = run(0, placement_neq, *goals_neq)

len(solutions_neq)

```

Out[31]: 92

```
In [32]: print_placement(solutions_neq[0])
```

```
Q . . . . .
. . . . Q . .
. . . . . Q
. . . . Q . .
. . Q . . . .
. . . . . Q .
. Q . . . . .
. . . Q . . .
```

3. 比较不同方法的运算时间、算法复杂度

本实验中，前面三种方法都可以求得八皇后问题的 92 个合法解。以本次 notebook 运行结果为例，各方法关键步骤的运行时间如下：

方法	思路	本次运行时间
方法一	非逻辑编程生成全排列，再用逻辑方式筛选	1.6 s
方法二	不预先生成候选解，直接使用逻辑编程求解	4.6 s
方法三	实现 <code>neq</code> ，并用 <code>neq</code> 表达不同列和不同对角线约束	15.8 s

方法一先生成 `1 ~ 8` 的所有全排列，共 $8! = 40320$ 个候选布局，再逐个筛选对角线约束。每个候选布局最多需要比较 $O(N^2)$ 对皇后，因此总体时间复杂度约为 $O(N! \cdot N^2)$

方法二不提前生成 `candidates`，而是直接使用逻辑变量和安全约束进行搜索。每确定一行皇后后，就检查它是否与之前的皇后冲突，因此可以较早排除不合法分支。不过，算法本质上仍然是在为 8 行皇后依次选择列号，并不断尝试可能的放置方案，所以搜索规模仍然与“穷举排列并筛选”的思路属于同一量级。

方法三使用自定义的 `neq` 函数表达“不相等”约束，将八皇后问题中的不同列、不同对角线约束都写成逻辑目标。该方法结构上更接近逻辑编程的表达方式，但由于约束数量较多，且每次判断都需要经过 Kanren 的目标函数和 `reify` 过程，因此本次运行时间最长。

(4) 使用BFS/DFS 解决八皇后问题，并与逻辑编程进行比较。

本小题使用传统搜索方法求解八皇后问题。与逻辑编程不同，BFS 和 DFS 都需要显式控制搜索过程：从空布局开始，逐步向下一行放置皇后，并在每一步检查当前布局是否合法。

其中，BFS 使用队列逐层展开状态；DFS 使用递归回溯深入搜索。两者都属于命令式/程式求解方法。

1. 公共合法性检查函数

```
In [33]: def is_safe_partial(placement, new_col):
```

判断在当前 `partial placement` 的下一行放置 `new_col` 是否安全。

```
placement[row] = col
当前要放置的行号为 len(placement)。
"""
new_row = len(placement)

for row, col in enumerate(placement):
    # 同列冲突
    if col == new_col:
        return False

    # 对角线冲突
    if abs(new_row - row) == abs(new_col - col):
        return False

return True
```

2. BFS 求解八皇后

```
In [34]: from collections import deque
```

```
In [35]: def solve_n_queens_bfs():
    """
    使用 BFS 求解八皇后问题。

    队列中的每个元素都是一个部分布局 placement。
    每次取出队首布局，尝试在下一行放置一个皇后。
    若新布局合法，则加入队列待后续展开。
    """
    queue = deque()
    queue.append([])

    bfs_solutions = []

    while queue:
        placement = queue.popleft()

        # 若已经放满 N 行，则得到一个完整解
        if len(placement) == N:
            bfs_solutions.append(placement)
            continue

        # 尝试在下一行放置皇后
        for col in nums:
            if is_safe_partial(placement, col):
                new_placement = placement + [col]
                queue.append(new_placement)

    return bfs_solutions
```

```
In [36]: bfs_solutions = solve_n_queens_bfs()

len(bfs_solutions)
```

```
Out[36]: 92
```

3. DFS 求解八皇后

```
In [37]: def solve_n_queens_dfs():
        """
        使用 DFS / 回溯法求解八皇后问题。

        每次递归负责确定下一行皇后的列号。
        若当前选择合法，则进入下一层；
        若放满 N 行，则记录一个完整解。
        """
        dfs_solutions = []
        placement = []

        def backtrack():
            if len(placement) == N:
                dfs_solutions.append(placement[:])
                return

            for col in nums:
                if is_safe_partial(placement, col):
                    placement.append(col)
                    backtrack()
                    placement.pop()

        backtrack()
        return dfs_solutions
```

```
In [ ]: dfs_solutions = solve_n_queens_dfs()

        len(dfs_solutions)
```

Out[]: 92

4. BFS / DFS 与逻辑编程比较

BFS 和 DFS 都是显式的搜索算法。程序需要自行控制搜索顺序、状态扩展和剪枝条件。BFS 按层展开所有部分布局，空间开销较大；DFS 沿着一条路径深入搜索，发现冲突后回溯，空间开销较小，因此更适合八皇后这类组合搜索问题。

逻辑编程方法则不直接描述搜索过程，而是声明变量取值范围和约束条件，由 Kanren 根据这些约束自动搜索解。它的表达方式更接近“描述什么是合法解”，而 BFS / DFS 更接近“说明如何一步步找到解”。

复杂度方面，BFS 和 DFS 本质上都需要在皇后放置方案中搜索。若不剪枝，搜索空间接近 N^N ；加入“不同列”和“对角线不冲突”的剪枝后，实际搜索规模明显减少。DFS 的空间复杂度约为 $O(N)$ ，BFS 需要保存大量同层节点，空间复杂度更高。逻辑编程的底层同样需要搜索，但代码层面更强调约束表达，而不是手动控制搜索过程。

(5) 写明调试过程，并提出有可能改进的方向。

1. 调试过程

本实验直接在 `.ipynb` 文件中按顺序执行各个代码块，并观察每个代码单元的输出结果。主要调试方式包括：

1. 检查每种方法最终得到的解的数量是否为 92；
2. 利用 notebook 自动显示的单元运行时间，比较不同方法的运行效率。
3. 通过 `print_placement` 函数直接打印布局，验证是否满足要求

在调试过程中发现，BFS、DFS 两种传统搜索方法运行很快，notebook 中显示的运行时间均为 `0.0s`；而逻辑编程相关方法耗时更长，尤其是使用自定义 `neq` 的方法，运行时间明显增加。

2. 可能的改进方向

1. 优化逻辑约束的加入顺序

在逻辑编程求解中，应尽量在变量取值确定后立即加入约束，使不合法分支尽早失败，减少无效搜索。

2. 减少 `reify` 调用次数

自定义 `neq` 和对角线约束中频繁使用 `reify` 会带来额外开销。可以尝试合并部分约束，减少重复取值和判断。

3. 改进逻辑编程表达方式

第三小题中使用 `neq` 后运行时间较长，说明逻辑目标数量和自定义目标函数开销较大。后续可尝试设计更紧凑的逻辑约束，或减少两两约束的重复判断。